OSI Compliant Data Transfer in ISM Band

Kim Hamberg, Mathias Huse, and Morten Sørensen
Department of Electronic Systems
Norwegian University of Science and Technology, Trondheim, Norway

***Abstract*—We implement a full-duplex Layer-1/2 radio link on ADALM-Pluto SDRs, exposed to Linux as a TUN interface. Operating in the 2.4 GHz ISM band with a self-built helix antenna, the system achieves 2.98 Mbit/s UDP throughput at 0.67% PER and 69 ms ping latency, supporting 1080p H.265 video, SSH, and unmodified TCP/UDP applications over the link.**

I. Introduction

In this report, we will implement a radio communication system and test it on Analog Devices Adalm Pluto software-defined radios (SDR) [1]. The goal is to achieve two-way communication in full-duplex mode compatible with the Linux TCP/IP stack, without accounting for moving nodes, and to achieve performance that enables streaming 1080p video using license-exempt bands in the 2.3 GHz to 2.5 GHz range, ensuring no interference with the university Wi-Fi.

II. Specification

The Open Systems Interconnection (OSI) model structures network communication into seven abstract layers [2]. This project implements Layers 1 and 2. The Linux kernel, via a TUN virtual network interface, implements Layers 3-7, exposing the radio as a network interface controller (NIC) carrying raw IP datagrams.

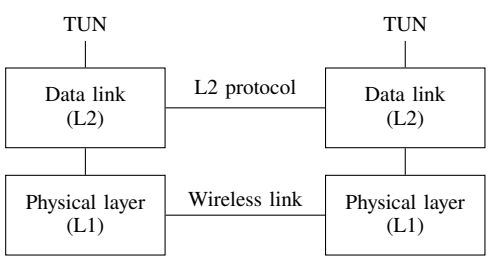

Fig. 1. OSI model adapted to this project. The OS Network Stack provides Layers 3-7 and connects to the radio via the TUN virtual interface. This project implements Layers 1-2.

The system will operate in two different ISM frequency bands under the *Fribruksforskriften* regulations on frequency use. Firstly §11.7, which permits data transfer in the band spanning from 2400.0 MHz to 2483.5 MHz at a maximum power spectral density of 10 mW MHz^{-1} e.i.r.p, when not using frequency-hopping spread spectrum (FHSS) [3]. Secondly, §17 for mobile video links, which permits video streaming in the bands 2327 MHz and 2390 MHz with a maximum radiated power of 2 W e.i.r.p, with a maximum occupied bandwidth of 8 MHz [3]. Given that the used radios' output power is well below 0 dBm, the system will operate below the threshold requiring FHSS for the first band and well below the maximum radiated power on the second band.

The required bit error rate (BER) applies to the raw link before data-link-layer retransmission, while the maximum packet error rate (PER) is the error rate after Automatic Repeat-Request (ARQ).

The SDR system pipeline is shown in Figure 2. Each frame from TUN uses the encoding shown in Figure 3, and the per-block parameters are in Table I and Table II. Encoding of "Type", "Mod.", and "Coding" fields of the header is in Table III. The ARQ works by parsing the header's type field. ACK frames reuse the same header with `Type = 01`, payload length 2 B, BPSK modulation, and an uncoded payload. The header's sequence-number field carries the cumulative ACK c – the highest sequence delivered in order, so $c + 1$ is the next expected (missing) sequence. The 16 bit payload is a big-endian selective-acknowledgment (SACK) bitmap covering the window past the gap: bit i marks sequence $c + 2 + i$ as buffered out-of-order at the receiver. The bitmap skips $c + 1$ because, by definition of the cumulative ACK, that sequence cannot already be buffered. A zero-length ACK is treated as bitmap = 0, falling back to Go-Back-N behavior. Packets whose type field is RAW bypass the ARQ entirely and are forwarded straight to TUN to keep overhead off that path.

For detecting packets and determining packet start, we utilize a full-buffer normalized cross-correlation of the received signal against the preamble Zadoff-Chu sequence. Peaks with a correlation magnitude above 0.3 are considered valid detections. From the cross-correlation, we also extract an estimated phase offset, which is passed to Costas for phase correction.

TABLE I
System Specifications.

Parameter	Value	Unit
RF		
Min frequency	2.405	GHz
Max frequency	2.478	GHz
Sample rate	6	MS s^{-1}
Samples/symbol	4	
Symbol rate	1.5	Msym s^{-1}
Pulse shape	RRC	
Filter span	8	sym
Filter roll-off	0.25	
Bandwidth	1.85	MHz

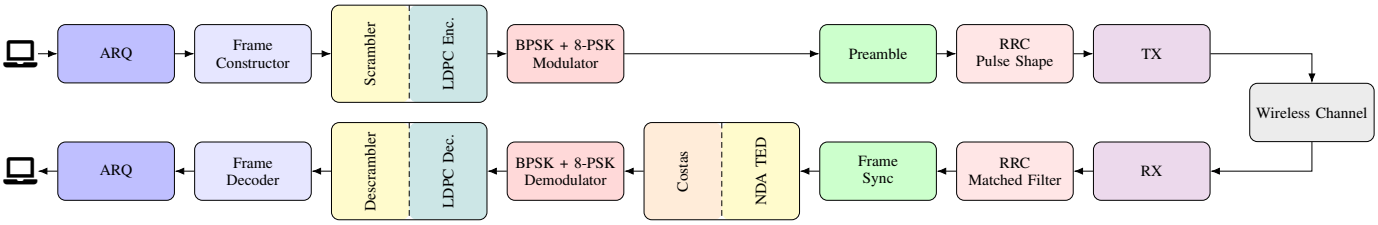

Fig. 2. Block diagram of the transmitter and receiver chain.

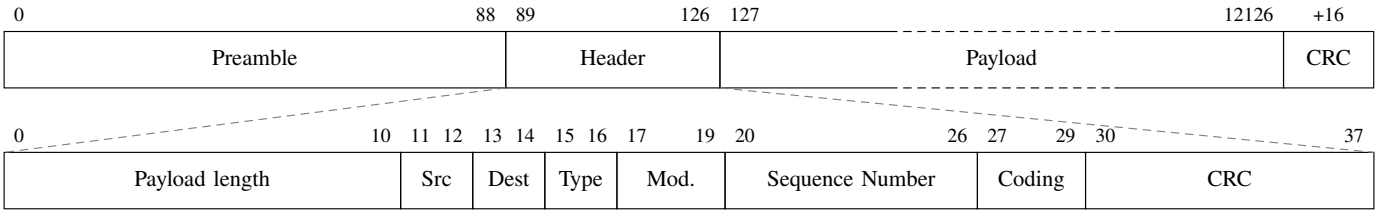

Fig. 3. Frame and header format. The horizontal dashed segments inside the payload denote the omitted middle bits; the diagonal dashed lines mark the expanded view of the 38 bit header. Src = source and Dest = destination.

TABLE II
SYSTEM SPECIFICATIONS CONTINUATION.

Parameter	Value	Unit
Frame format		
Header length	38	bit
Header coding	uncoded	
Header CRC	CRC-8	
Header CRC polynomial	0x07	
Payload length	1 to 1500	B
Payload coding	LDPC	
Payload CRC	CRC-16-CCITT	
Payload CRC polynomial	0x1021	
Scrambler RNG seed	0xC3C3C3C3	
Scrambler length	16	
LDPC rates	X, 1/2, 2/3, 3/4, 5/6	
LDPC block length	1944	bit
LDPC convention	IEEE Std 802.11 [4]	
Preamble		
Type	Zadoff-Chu	
Root	13	
Length	89	
Modulation		
Header scheme	BPSK	
Payload scheme	8-PSK	
ARQ		
Algorithm	Selective Repeat	
Window size	63	
Performance		
Min data rate	1	$\mathrm{Mbit\,s^{-1}}$
Min range	10	m
Max end-to-end latency	300	ms
Max jitter	50	ms
Max PER	1	%

TABLE III
ENCODING OF THE TYPE, MOD., AND CODING HEADER FIELDS, TRANSMITTED MSB-FIRST. RAW FRAMES BYPASS ARQ (E.G. UDP); CTRL IS RESERVED. UNLISTED VALUES ARE RESERVED.

Type (2 bit)		Mod. (3 bit)		Coding (3 bit)	
00	DATA	000	BPSK	000	uncoded
01	ACK	001	QPSK	001	1/2
10	RAW	010	8-PSK	010	2/3
11	CTRL	011	16-PSK	011	3/4
				100	5/6

III. LINK BUDGET

Based on the specifications, we drafted a link budget using hardware measurements, International Telecommunication Union (ITU) channel models to estimate path loss, and assumptions for various other losses. The link budget is in Table IV, which uses the center frequency of $f_0 = 2.478\,\mathrm{GHz}$ rather than a separate budget for each carrier. Since they are closely spaced, their impact on the link budget is negligible.

We derive the required E_b/N_0 from BER = 10^{-5}. The TX power P_{tx} was measured, falling substantially short of what its nominal specifications would suggest, raising questions about its utility beyond introductory pedagogical contexts. We characterized antenna gain G_A for the self-made helix antenna at the NTNU antenna lab, and measured the receiver noise temperature T in lab C430 at f_0. The two connector losses are not in the link budget, since the measurement of P_{tx} and G_A accounts for both. The receiver noise figure (NF) is the worst-case figure from the data sheet [5]. We calculated channel loss L_{channel} using the general site model ITU-R P.1238-13 [6, p. 6] with Office LOS parameters: $\alpha = 1.47$, $\beta = 34.17$, and $\gamma = 2.08$. The measured E_b/N_0 and the verification of the link budget are presented in Section V-B.

TABLE IV
LINK BUDGET. BOLDFACED ROWS ARE DERIVED FROM THE UNBOLDFACED INPUTS ABOVE THEM.

Parameter	Value	Unit
Transmitter		
P_{tx}	−7.0	dBm
G_A	10.5	dBi
EIRP	**3.5**	**dBm**
Propagation		
D	3.0 / 10.0	m
$\boldsymbol{L_{\text{channel}}}$	**49.4 / 57.1**	**dB**
Receiver		
$\boldsymbol{G_A}$	**10.5**	**dB**
T	61.9	dB K
NF	3.5	dB
$\boldsymbol{G/T}$	**−54.9**	**dB K^{-1}**
$\boldsymbol{N_0}$	**−133.2**	**dBm Hz^{-1}**
Performance		
$\boldsymbol{R}$	**64.8**	**dB Hz**
Calculated		
$\boldsymbol{E_b/N_0}$	**33.1 / 25.4**	**dB**
Measured		
$\boldsymbol{E_b/N_0}$	**31.7 / –**	**dB**
Link		
Req. E_b/N_0	13.3	dB
G_C	6.7	dB
Link margin	**26.5 / 18.8**	**dB**

IV. DESIGN MOTIVATION AND IMPLEMENTATION

A. Programming tools

We implemented the pipeline in Python with C++ pybind11 extensions for the DSP-blocks. This combination lets us easily verify and test behavior with Python and Pytest while achieving speed with C++. Claude Code [7] was used for porting Python code to C++.

B. Frequency band

We chose the 2.4 GHz ISM band (2400.0 MHz to 2483.5 MHz), license-free under EN 300 328, for three reasons. First, the band imposes no duty-cycle restriction, unlike sub-GHz alternatives such as 868 MHz, where a 1 % duty cycle directly bounds throughput. Second, the 83.5 MHz of contiguous spectrum comfortably accommodates the 1.85 MHz channel. Lastly, for our output power, no FHSS is required. Additionally, the video link band 2327 to 2390MHz was chosen because it is less congested than the 2.4 GHz band and has fewer regulatory requirements. As one of the goals is to transmit video, this band is preferable to the 2.4 GHz band.

C. Modulation schemes

The header uses BPSK modulation because a header-decoding failure invalidates the entire frame, as the receiver loses the payload length and the modulation indicator needed to demodulate the rest of the message. Also, BPSK requires the lowest E_b/N_0 of any phase modulation. The header is short enough (38 bit) that the throughput cost of BPSK over a higher-order scheme is negligible. The payload uses 8-PSK modulation for two reasons. First, this enables us to use the Costas Loop, which we want because of the reasons listed in subsection IV-J. Second, maintaining the constant-envelope property of PSK, which Pluto's TX power amplifier strongly favors. The header includes a field for modulation order because we do not use 8-PSK modulation for ACK packets. Instead, we use BPSK to ensure the ACK is decoded correctly, thereby maintaining ARQ operability. Furthermore, the inclusion would enable future iterations to use dynamic modulation order based on channel loss, using the 4th, now unused, packet type for back-channel control, which was the original intent.

D. Data frame structure

The frame format (Figure 3) draws inspiration from IEEE 802.11b: a fixed-length header containing length, addressing, type, modulation, sequence, and integrity fields, followed by a variable-length payload. This layering lets the receiver decode the header at a robust modulation (BPSK), learn the parameters needed to demodulate the rest of the frame, and then proceed.

The 38 bit header is divided as follows. The 11 bit payload-length field covers up to 2048 B, comfortably above the 1500 B maximum from Table I. Source and destination addresses are 2 bit each, enabling up to 4 nodes. A 3 bit distinguishes between three different packet types: payload packets, ACK packets, and UDP packets. ACK packets are structured as discussed in section II. UDP packets do not go through ARQ, so they have a specific frame type. The 2 bits support one more packet type, but are reserved for future improvements. A 3 bit modulation field encodes the payload constellation, and the field's width leaves room for adaptive modulation extensions without a header change. A 7 bit sequence number supports a 64-frame ARQ window. Following with 3 bit specifying coding rate. The header closes with an 8 bit CRC over the preceding fields.

The frame is a Layer-2 (data-link) construct. The receiver reads and acts on every header field: it uses the payload-length field to locate the frame boundary, the destination bit to decide whether the frame is for this node, the type field to dispatch to data- or ARQ-handling code, the sequence number to track retransmissions, and the header CRC to discard frames when the wireless channel corrupts the control bits before any payload demodulation begins. The layer never inspects the payload itself. It hands it up the stack through the TUN interface, where the Linux kernel parses it and handles it from there. This follows the OSI principles [2, §§ 5.6.1.3, 5.6.1.4]: each layer wraps the data from above with its own header (the protocol-control-information), and on receive reads only that header before passing the rest up as a service-data-unit, which the standard explicitly defines as information "not interpreted by the supporting (N)-entities" [2, § 5.6.1.4].

E. Source coding

The payload that crosses the air interface is the raw IP datagram produced by the Linux kernel, and the application

layer that produces it is assumed to have already compressed its own data (e.g., H.265 for video and MP3 for audio). A second, system-level source coder would therefore be redundant.

F. Channel Coding

We apply channel coding only to the payload. The header is transmitted uncoded, with the 8-bit CRC at its tail providing error detection without correction. Two arguments support the choice. The header modulation scheme is BPSK, the most robust phase modulation, so its raw bit-error rate at the operating SNR is low without any added redundancy. Any uncorrected header errors result in a CRC failure and frame discard, which the data-link-layer ARQ then recovers. Adding FEC over a 30-bit field would therefore reduce throughput without meaningfully reducing frame loss as long as ARQ works as intended.

For the payload, we use the LDPC codes specified in IEEE 802.11n [4] at rate $R = 5/6$, with block lengths of 648, 1296, or 1944 bit. The choice of $R = 5/6$ is appropriate because the link margin in Table IV remains substantial even at this rate, and one of the project's primary goals is raw throughput. At $R = 5/6$, the corresponding per-frame information capacities are 540, 1080, and 1620 bit, so the maximum payload per frame is 202 B when the largest block size is selected, which is well below the 1500 B specification maximum from Table I. By setting the TUN interface's MTU to match the IP datagram size, we accommodate larger IP datagrams, so the kernel produces appropriately-sized datagrams without relying on data-link-layer fragmentation.

For LDPC decoding, we utilize soft decoding. Instead of demodulating symbols directly, we decode bits into Log Likelihood Ratio (LLR) values by calculating a normalized min-sum with $\alpha = 0.75$ per symbol. The use of soft decision improves coding gain by 2 to 3 dB over hard decision [8], normalized min-sum eaking slight performance improvements over normal min-sum. Since these improvements do not require system architecture changes, the additional coding gains are well worth the extra compute time for LLR.

G. Scrambling

To mitigate errors caused by real-world data being correlated, a scrambler was added. We generate a 2^{16} bit sequence with seed 0xC3C3C3C3. This sequence is XOR-ed against our data before sending it to LDPC for channel coding. This whitens the data, making the rest of our system less susceptible to the adverse effects of constant signals, such as odd behavior in Pluto's DC filter and timing sync drift.

H. Preamble

The method used to detect preambles works with many types of signals. The particular properties of the Zadoff-Chu sequence have motivated us to use it. A preamble with a flat frequency envelope (spread signal energy) will be more robust to channel interference than a preamble with a non-flat envelope (concentrated signal energy). Note that white noise also has a flat envelope. Therefore, to detect the preamble, we use its known phase structure via matched filtering. Additionally, a preamble with constant amplitude lets AGC converge to a fixed gain and hold it. This avoids a signal-dependent gain that would otherwise distort the preamble and reduce the correlation peak. We chose to use Zadoff-Chu, which has both properties (CAZAC). Also, 3GPP uses ZC for similar reasons in LTE's PSS, where the UE needs to detect PSS transmitted from the BS under unknown timing, unknown CFO, and possibly low SNR.

I. Symbol Synchronization

For symbol synchronization, a non-data-aided symbol timing recovery method was used. The implementation is derived from the `NDA_symb_sync` function in the scikit-dsp-comm Python package [9]. The algorithm exploits the periodicity of the squared signal magnitude $|z|^2$ at the symbol rate. This makes it inherently modulation-order independent for M-PSK signals. The DFT coefficient at frequency $1/SPS$ of $|z|^2$ is zero when timing is perfect and non-zero otherwise, providing a timing error signal without any knowledge of the transmitted data. We chose this because it is robust and stable for our modulation schemes.

J. Phase correction and tracking

Since the project's goal is to have a high data rate and reduced latency, we chose a Costas loop for phase correction and tracking. As it is non-data-aided and uses only the known constellation points, it introduces no additional payload overhead. Computationally, it is also quite light, which is ideal to reduce RX computation time. On its own, the Costas loop suffers from phase ambiguity, where if the constant phase offset is more than half the angular distance to the next constellation point, it will lock onto the wrong phase. Because the phase estimate from the Zadoff-Chu preamble is accurate and unambiguous, it can serve as a starting point for the Costas loop, eliminating this issue. The Costas implementation for BPSK and QPSK is based on the structure shown in [10], while our 8-PSK Costas implementation is based on the GNU Radio implementation [11].

K. Antenna choice

Operating on the 2.4 GHz ISM band means there will be interference from Wi-Fi. To mitigate this, we use a helix antenna rather than a standard dipole antenna. It gives significantly better directivity and gain. Additionally, it has circular polarization, which is more robust against multipathing because the polarization reverses direction upon reflection. This results in the receiver picking up only a multipath signal that has bounced an even number of times. As it is fairly simple to make and the availability of 2.4 GHz helix antennas was not great, we made our own antenna using 3D-printed parts and simple copper wire, after consulting with Egil Eide.

V. Measurements and verification

A. Regulatory compliance

To validate regulatory compliance, we performed the power spectral density measurement described in section 5.4.3.2

of ETSI EN 300 328 V2.2 for non-FHSS equipment with continuous transmission capability [12].

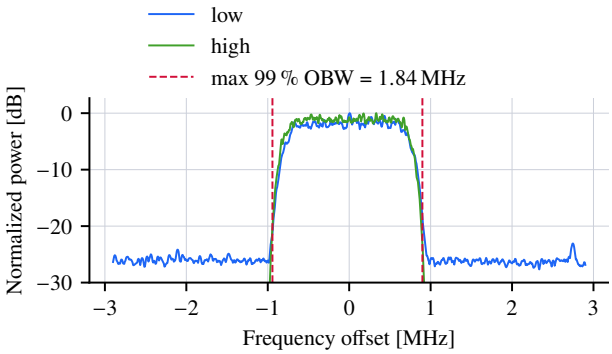

Fig. 4. Peak-normalized PSD at the low (2.4050 GHz) and high (2.4780 GHz) channel centers. Dashed lines mark the larger of the two 99 % occupied bandwidths per §4.3.2.7 of [12].

A maximum PSD of −7.4 dBm was measured at 2.478 GHz. The video link band does not have an ETSI standard for measurements, but we measured to be below −7 dBm.

We characterized the helix antenna in the NTNU antenna lab from 2 to 3 GHz against a transmitting reference antenna at 5 m. After free-space path loss, cable loss, reference-antenna gain, and 3 dB polarization correction, the realized helix gain is $G_A = 10.5$ dBi at 2.478 GHz. For the video link band, the realized antenna gain is lower at $G_A = 8.6$ dBi. Combined with the measured PSDs, the radio remains compliant throughout transmission for both frequency bands.

While configuring the radio, a power burst was observed during sample rate setting. This peak varies widely from Pluto to Pluto and was measured above 0 dBm on some radios, meaning it is not within the regulations on boot for the 2.4 GHz band. This is not an issue for the video link band, as the regulations permit a higher radiated power. This burst is shown in Figure 7.

B. System performance

Raw system performance was measured by sending synthetic packets containing random data to obtain results independent

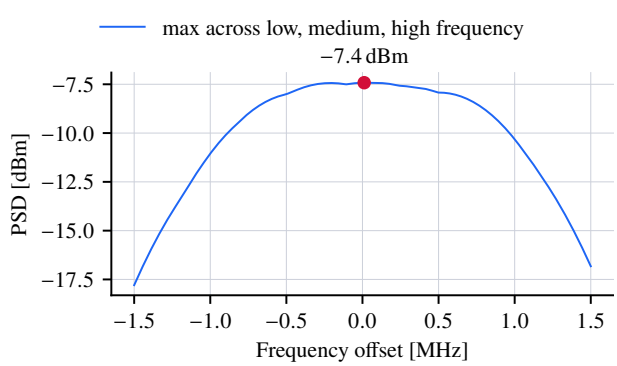

Fig. 5. Pointwise maximum of three captures at the low (2.4050 GHz), medium (2.4415 GHz), and high (2.4780 GHz) channel centers per §5.4.3.2 of [12].

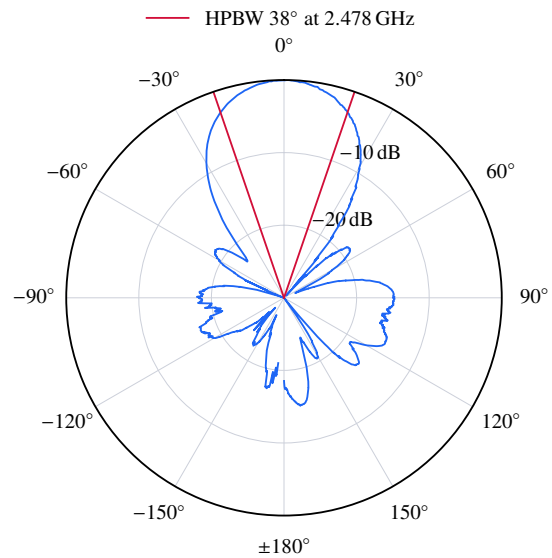

Fig. 6. Measured normalized antenna radiation pattern ($N = 25$) clipped at −30 dB. We apply a Savitsky-Golay filter (1° window, cubic polynomial), which preserves peak amplitude but suppresses measurement jitter.

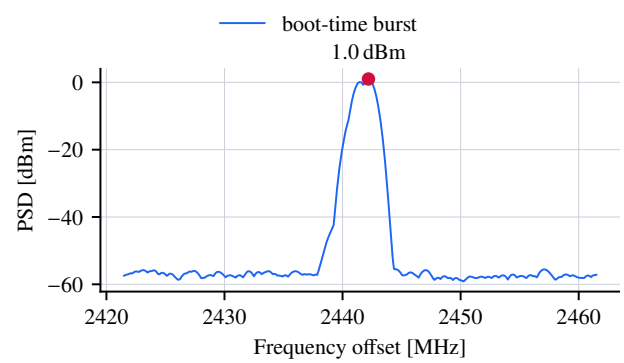

Fig. 7. Trace of emitted boot-time burst during the time Pluto is setting the sample rate.

of the transmitted data. Since we are using a scrambler, the results should be equivalent to those for real data. The received constellation diagram with Error Vector Magnitude (EVM), over a 3 m link inside C430, is shown in Figure 8.

With the EVM metric, the channel's E_b/N_0 can be calculated. This is shown in Table V.

TABLE V
EVM, CHANNEL E_b/N_0, 90TH AND 99TH PERCENTILE.

EVM (RMS)	E_b/N_0	p50	p99
0.026	31.7 dB	34.31 dB	23.51 dB

The link budget is therefore confirmed at $D = 3$ m.

C. Data transfer TCP/UDP

For two-way communication, two plutos were used on each computer, one for RX and one for TX, due to limitations of the USB 2.0 interface. TCP is tested via ping to verify it is

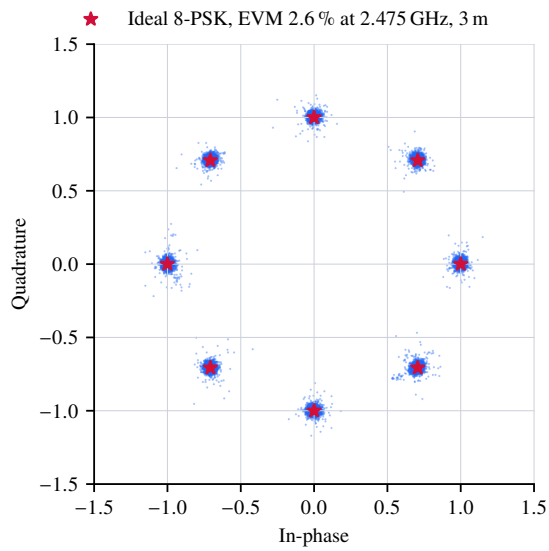

Fig. 8. Received 8-PSK constellation captured at 2.475 GHz over a 3 m indoor link. We calculated the RMS EVM from 3.3 million decoded symbols.

working correctly and that the max end-to-end latency meets the specifications. Ping performance is shown in Table VI.

TABLE VI
SHOWS WORKING PING FROM ONE COMPUTER TO ANOTHER THROUGH THE RADIO LINK.

Min	Avg	Max	Stdev	Unit
59.678	69.294	86.052	7.313	ms

Ping performance is stable and well within specifications.

For UDP datarate, iperf was used to generate messages and measure throughput. The link was tested at several coding rates, with the results shown in Table VII.

TABLE VII
SHOWS UDP TRANSFER SPEED OVER RADIO LINK AT ALL CODING RATES.

Coding Rate	Total (MB)	Rate (Mbit s^{-1})	Jitter (ms)	Lost/total
None	9.35	2.58	8.154	1130/8135 (14 %)
5/6	10.8	2.98	7.161	55/8151 (0.67 %)
3/4	10.8	2.98	7.163	59/8147 (0.72 %)

This result verifies the need for channel coding but shows no measurable difference between the 5/6 and 3/4 rates. With TCP and UDP confirmed to work, the system can be used for every use case that works over a standard wired Ethernet connection. We tested this by playing on a local Minecraft server over the radio link and by connecting to the other computer over SSH.

D. Video streaming

An mp4 video was streamed using ffmpeg with h.265 codec and 3s of buffering. We measured the received video quality using the following performance metrics: PSNR, SSIM, and VMAF with `ffmpeg-quality-metrics`. We used the 2.39 GHz band for video streaming to avoid Wi-Fi interference. The resulting quality metrics are presented in Table VIII. Visual inspection of the video stream also presented excellent results.

TABLE VIII
VIDEO QUALITY METRICS

Metric	Value	Range
PSNR	30.9 dB, stdev 1.1	dB
SSIM	0.952 avg, min 0.938	0 to 1
VMAF	62.7 avg, stdev 4.8	0 to 100

In addition to streaming an MP4 video, the link was tested by streaming a webcam feed with audio, functioning as a video call.

VI. DISCUSSION

Regarding the need for two radios per user to achieve two-way communication, this was a limitation of the libiio library used to interact with the devices. The library's rx and tx calls do not handle threading automatically, leading to operations being blocked if not explicitly handled further up in the program. We did not have time to figure out how to handle this, so we settled on using separate radios for RX and TX to accomplish the task. Future work would be to figure out the libiio limitations or to deal with them in the code handling transmission and reception.

To achieve the current throughput with two-way communication on a single Pluto, the main limitation would be the Pluto's USB 2.0 interface. A possible solution would be to move upsampling, pulse shaping, and filtering to the FPGA on the TX side to reduce USB throughput, since the data sent to the Pluto for TX could be 1/SPS less. On the RX side, the filtering could also be implemented in the FPGA, and potentially the frame sync as well, so the only data sent to the computer over USB are buffers with detections.

VII. CONCLUSION

We implemented a Layer-1/2 radio link on Adalm Pluto SDRs, exposed to Linux as a TUN interface, that meets the functional and performance targets specified in Table II. The PSD measurement verified compliance with ETSI EN 300 328, the measured E_b/N_0 at 3 m confirms the link budget. UDP throughput reached 2.98 Mbit s^{-1} at $R = 5/6$ LDPC with PER below 0.7 %. Ping latency averaged 69 ms, well inside the 300 ms target. End-to-end usability demonstrated 1080p H.265 video streaming, live webcam video call, and unmodified TCP/UDP applications (SSH, Minecraft) running over the TUN interface. Two limitations remain: Pluto's transmit power falls substantially short of its nominal specification, and a boot-time burst can briefly exceed the 2.4 GHz ISM limit on some units. The reserved fourth packet type and the modulation-rate header field leave room for a future adaptive-modulation back-channel.

References

[1] Analog Devices, Inc. ADALM-PLUTO evaluation board. Evaluation Hardware and Software. Norwood, MA, USA. Accessed: 2026-04-25. [Online]. Available: https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/adalm-pluto.html

[2] *Information Technology — Open Systems Interconnection — Basic Reference Model: The Basic Model*, ITU-T ITU-T Recommendation X.200, Jul. 1994. [Online]. Available: https://www.itu.int/rec/T-REC-X.200-199407-I/en

[3] Samferdselsdepartementet. (2012, Jan.) Forskrift om generelle tillatelser til bruk av frekvenser (fribruksforskriften). Norwegian Regulation FOR-2012-01-19-77. Oslo, Norway. Accessed: 2026-04-25. [Online]. Available: https://lovdata.no/dokument/SF/forskrift/2012-01-19-77

[4] *IEEE Standard for Information Technology — Telecommunications and Information Exchange between Systems — Local and Metropolitan Area Networks — Specific Requirements — Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications*, IEEE IEEE Std 802.11-2020, Rev. Revision of IEEE Std 802.11-2016, Feb. 2021. [Online]. Available: https://doi.org/10.1109/IEEESTD.2021.9363693

[5] Analog Devices, Inc. ADALM-PLUTO SDR active learning module. Product Highlight. Norwood, MA, USA. Accessed: 2026-04-25. [Online]. Available: https://www.analog.com/media/en/news-marketing-collateral/product-highlight/adalm-pluto-product-highlight.pdf

[6] *Propagation Data and Prediction Methods for the Planning of Indoor Radiocommunication Systems and Radio Local Area Networks in the Frequency Range 300 MHz to 450 GHz*, ITU-R ITU-R Recommendation P.1238-13, Sep. 2025. [Online]. Available: https://www.itu.int/dms_pubrec/itu-r/rec/p/R-REC-P.1238-13-202509-I!!PDF-E.pdf

[7] Claude code overview - claude code docs. Claude Code Docs. [Online]. Available: https://code.claude.com/docs/en/overview

[8] R. Jose and A. Pe, "Analysis of hard decision and soft decision decoding algorithms of LDPC codes in AWGN," in *IEEE International Advance Computing Conference (IACC)*, 2015, pp. 430–435.

[9] M. Wickert. Source code for sk_dsp_comm.synchronization. scikit-dsp-comm Documentation. Accessed: 2026-04-25. [Online]. Available: https://scikit-dsp-comm.readthedocs.io/en/latest/_modules/sk_dsp_comm/synchronization.html#NDA_symb_sync

[10] Q. Chaudhari. (2022, Apr.) Costas loop for carrier phase synchronization. Wireless Pi. Accessed: 2026-04-25. [Online]. Available: https://wirelesspi.com/costas-loop-for-carrier-phase-synchronization/

[11] K. Krishna. 8-PSK-Costas-Loop. GitHub repository. Accessed: 2026-04-25. [Online]. Available: https://github.com/martiansideofthemoon/8-PSK-Costas-Loop

[12] *Wideband Transmission Systems; Data Transmission Equipment Operating in the 2,4 GHz Band; Harmonised Standard for Access to Radio Spectrum*, European Telecommunications Standards Institute ETSI EN 300 328, Rev. V2.2.2, Jul. 2019. [Online]. Available: https://www.etsi.org/deliver/etsi_en/300300_300399/300328/02.02.02_60/en_300328v020202p.pdf

VIII. GitHub Repository

https://github.com/MortenSorensenNOR/TTT4145-Semester-Project